

RISC-V LINUX 嵌入式

编程技术 · 第二章

工具链与工程

host / target / sysroot → 交叉编译 → ELF 分析

Makefile 工程化 → 统一目录规范

— 2.1 / 2.2 / 2.3 —

工程模板: hello + project-
template

工具链: riscv64-unknown-linux-
gnu-gcc

开始之前

前置条件：本章假设你已完成第一章全部内容——ruiy 已安装、工具链已验证、开发板已启动且 SSH/SCP 通道可用。每节约需 90–120 分钟（含本章 `lab.html`）。

你需要准备什么

主机

Linux
x86_64,
已安装
ruiy 与
RISC-V
GNU
Toolchain
(交叉编
译器前缀
`riscv64-
unknown-
linux-
gnu-`)。

开发板

LicheePi
4A,
RevyOS
已启动,
SSH/SCP
免密登录
可用。本
章所有代
码在板端
运行验
证。

代码

课程仓库
`chapters/ch02/code/`
下的 `hello` 和 `project-
template` 工程模板。

本章你会学到什么

2.1 交叉编译全流程

理解
host/target/sysroot, 使用
交叉编译器构建第一个
RISC-V 程序, 部署到板
端运行并用 `readelf/ldd` 分
析二进制结构。

2.2 Makefile 与工程目录

编写 Makefile 规则与变
量, 理解增量构建与
clean, 掌握
`src/include/build` 统一目录
规范——后续实验的标准
工程模板。

2.3 CoreMark 算力基准

了解 CPU 基准测试的用途与局限；动手跑分见本章 `lab.html`。

学习建议：2.1 → 2.2 是递进关系——先用 hello 理解单文件编译部署，再用 project-template 掌握多文件工程化。每个操作步骤都有对应的验证方法。完整终端日志是实验验收材料——从现在开始养成保存日志的习惯。

交叉编译全流程：从源码到板端可执行文件

学习目标

host / target /

sysroot

区分编译主机、运行设备和目标根文件系统，理解交叉编译

工具链查询

确认交叉编译器目标三元组，定位 sysroot 路径

编译 → 传输 → 运行

使用 hello 工程完成首个交叉编译闭环

ELF 分析

使用 readelf / ldd 查看二进制结构与动态库依赖

架构验证

用 file + readelf 确认产物为 RISC-V ELF

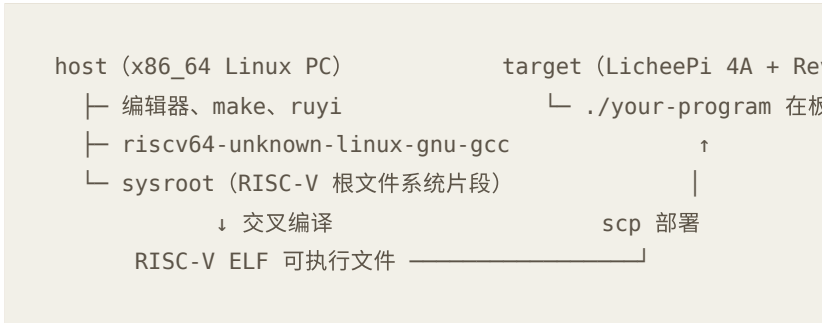
两端验收

区分 host 编译成功与 target 运行成功两个验收环节

host、target、sysroot 与交叉编译

嵌入式 Linux 开发很少在开发板上直接编译大型程序。更常见的流程是：在性能更好的 PC（host）上生成二进制，再部署到开发板（target）执行。当 host 与 target 的 CPU 架构不同时，称为**交叉编译**（cross compilation）。

工具链除了编译器，还需要目标系统的头文件和库。这些文件按目标系统的目录结构组织，合称为 **sysroot**。编译时编译器在 sysroot 中查找头文件和库；链接时按目标 ABI 链接其中的库。



环境准备

项目	要求	检查方式
host 架构	记录主机 CPU 架构	<code>uname -m</code>
target 架构	板端为 RISC-V 64 位	<code>ssh user@board- ip uname -m</code>
交叉 编译 器	前缀 <code>riscv64-unknown-linux- gnu-</code>	<code>command -v riscv64- unknown-linux- gnu-gcc</code>
工程 目录	<code>chapters/ch02/code/hello/</code>	<code>ls main.c Makefile</code>
分析 工具	host 有 <code>readelf</code> 、 <code>file</code>	<code>command -v readelf</code>

关键区分： `riscv64-unknown-elf-gcc` 面向无操作系统环境（裸机）。本课程用户态程序应使用带 `linux-gnu` 的工具链。两者不可互换。

操作步骤

- 1

确认 host 与 target

主机 `uname -m` (x86_64)，板端 `uname -m` (riscv64)
——两者不同说明需要交叉编译。
- 2

查看交叉编译器目标三元组与 sysroot

`riscv64-unknown-linux-gnu-gcc -dumpmachine` 应含

riscv64 和 linux-gnu。-print-sysroot 查 sysroot 路径。

3 对比 host 与 target 运行环境

两端分别执行 file /bin/bash 和 ldd /bin/bash | head -n 3，观察动态链接器路径与架构差异。

4 交叉编译第一个程序

cd chapters/ch02/code/hello && make clean && make && file hello — 应显示 RISC-V ELF。

5 部署到板端并运行

scp hello user@board-ip:~/ && ssh user@board-ip './hello' — 预期输出 Hello, RISC-V Linux!

6 ELF 分析

readelf -h hello (Machine=RISC-V)、readelf -l hello (LOAD 段)、ldd hello (动态库依赖或注明静态链接)。

ELF 关键字段

字段	含义	期望
Class	ELF 位数	ELF64
Data	字节序	小端，LSB
Machine	CPU 架构	RISC-V
Type	文件类型	EXEC 或 DYN

为什么不在主机验证：x86_64 主机无法执行 RISC-V 二进制。 ./hello 在主机必然报错——正确验收是在板端执行。用 file 确认架构，用板端运行确认逻辑。

课堂练习

1. 「在 x86_64 笔记本上 gcc hello.c 得到的程序，可以直接 scp 到 LicheePi 4A 运行。」——这句话是否正确？为什么？

2. 若 `scp` 成功但板端提示 `cannot execute binary file` , 依次检查哪些项目?
3. `readelf -h` 中 `Machine` 显示 `X86-64` 时, 构建过程哪里出了问题?

本节成果：host/target/sysroot 对照表 · 交叉编译器三元组与 sysroot 路径 · 板端可运行的 Hello World · 可复用的「编译 → scp → 运行」命令链 · readelf/ldd 分析记录。

Makefile 与工程目录规范

学习目标

规则与依赖

理解目标、依赖、命令三要素与 Tab 缩进

变量管理

使用 CC、CFLAGS、CROSS_COMPILE 避免硬编码

增量构建

make 时间戳判断 + clean + .PHONY

目录约定

src/ (源码) + include/ (头文件) + build/ (产物分离)

多文件链接

模式规则将多 .c 编译到 build/ 下统一链接

工程复用

模板可扩展至后续 GPIO、MQTT、综合项目

Makefile 基础

Makefile 描述「什么文件由什么文件生成、执行什么命令」。

`make` 根据文件时间戳决定是否重新编译。

`chapters/ch02/code/hello/Makefile` 是单文件示例，`project-template/Makefile` 在此基础上增加多源文件与 `build/` 目录。

```
Makefile
CROSS_COMPILE, CC, CFLAGS
↓
hello: main.c
    $(CC) $(CFLAGS) -o $@ $^
```


↓
make / make clean

统一工程目录

单文件 `hello` 适合入门，真实项目应将代码按职责分离：

```
project-template/  
├─ Makefile  
├─ include/      ← 对外头文件 (greet.h)  
├─ src/          ← 源文件 (main.c + greet.c)  
└─ build/        ← 产物 (.o + app)，可删可重建
```

编译流程：

```
src/main.c ─┐  
src/greet.c ┤─> build/src/*.o ─> build/app  
include/ ─┘
```

操作步骤

阶段 A：单文件 Makefile

```
$ cd chapters/ch02/code/hello  
$ cat Makefile  
  
# 完整构建流程  
$ make clean && make  
$ make                # 第二次应提示 up to date  
$ touch main.c  
$ make                # 应重新编译  
  
# 清理  
$ make clean && ls hello 2>&1  # hello 应已删除  
  
# 命令行覆盖变量  
$ make CFLAGS='-Wall -Wextra -O0 -g'
```

课堂提示：跟做时可在 `main.c` 临时加入未使用变量，观察 `-Wall` 警告。**Makefile 命令行必须以 Tab 缩进**，用空格会触发 `missing separator`。

阶段 B：多文件工程

```
$ cd chapters/ch02/code/project-template
$ find . -type f | sort

$ make clean && make
$ file build/app                # RISC-V ELF

# 部署并运行
$ scp build/app user@board-ip:~/
$ ssh user@board-ip 'chmod +x ~/app && ./app'
# 预期输出: Hello, RISC-V Linux!

# 清理重构
$ make clean && test ! -d build && echo "build removed"
```

工程习惯： build/ 应加入 .gitignore（可删除、可重建），src/ 和 include/ 纳入版本库。不要把 .o 文件提交到 Git——它们是派生产物，不是源码。

课堂练习

1. 以下 Makefile 错误分别导致什么现象：命令行前用空格而非 Tab；目标未声明依赖且产物已存在；`clean` 未标记 `.PHONY` 且目录下有同名文件？
2. 为何 `build/` 加入 `.gitignore`，`src/` 和 `include/` 纳入版本库？

本节成果： Makefile 变量与规则解释 · make / make clean / 增量构建演示 · 符合 src/include/build 约定的可运行工程 · 板端运行 app 记录 · 可复用模板路径
chapters/ch02/code/project-template/。

CoreMark 算力基准

学习目标

基准认知

CoreMark 测什么、不测什么

分数解读

CoreMark 分数与 CoreMark/MHz 的含义与局限

与课程关系

算力直觉如何帮助评估后续算法与外设实验，而非替代真实场景测试

CoreMark 是什么

CoreMark 是 EEMBC 发布的轻量级 CPU 基准，主要覆盖列表、矩阵、状态机等核心逻辑。结果常以绝对分数或 CoreMark/MHz 报告。它反映 CPU 纯算力，**不能**代表 GPIO 时序、网络、存储 I/O 等系统能力。

为什么要学

交叉编译与 Makefile 解决「程序能跑」；CoreMark 帮助建立「这块板子算力大概什么量级」的直觉，便于判断软解码、图像处理等是否现实。本章动手在板端跑分——步骤在 `lab.html`。

理解分数时注意

- 编译器版本、`-O` 级别、glibc/musl、CPU 频率都会影响结果。
- 只宜同环境前后对比，不宜盲目攀比网上截图。
- MQTT、传感器类实验是否流畅，还取决于 I/O 与延迟，不能只看 CoreMark。

课堂练习

列举三项 CoreMark **无法单独反映**、但在本课程实验中很重要的系统能力。

本节成果：能口头说明 CoreMark 用途与局限 · 板端跑分与简短评述见 `lab.html` 。